arXiv:2611.09722v1 [cs.AI] 6 Nov 2026

FOSCAR: ACCELERATING DECODE ATTENTION OVER 2- AND 4-BIT KV CACHES THROUGH AFFINE FOLDING

Donglin Zhuang [1 2] **Pragaash Ponnusamy** [1] **Xiaoxia Wu** [1] **Shuaiwen Leon Song** [2] **Zhongzhu Zhou** [1 2]

ABSTRACT

Large-language-model decode attention streams the key–value (KV) cache for every generated token. Packing that cache into 2-bit or 4-bit quantized integers reduces memory traffic but does not automatically translate into latency saving. Existing 2-bit fused kernels only reach less than 31%/15% of peak HBM throughput on H100/B200. We identify that converting each value and applying its scale and offset before feeding Tensor Cores is a major bottleneck because instruction-issue throughput is a scarce resource on modern GPUs. FOSCAR does not apply scale and offset to each value from the KV cache, but uses *affine folding* to apply scale and offset to either the other operand or the output of matrix products in decode attention, depending on the quantization scheme (token-wise or channel-wise). This reduces scale-and-offset operations by up to two orders of magnitude while preserving the dequantized attention output in exact arithmetic. FOSCAR includes carefully designed bit manipulation to feed KV operands to Tensor Cores efficiently, requiring only 1 and 0.19 ALU instructions per element for the INT2 KV cache on H100 and B200, respectively. Finally, FOSCAR builds warp-specialized GPU pipelines for Hopper (H100) and Blackwell (B200), leveraging architecture-specific features such as `wgmma` and `tcgen05`. On H100, FOSCAR INT2 and INT4 kernels achieve geo-mean speedups of 3.36× and 2.83× over FlashInfer BF16, and 2.03× and 1.71× over TensorRT-LLM FP8. On B200, the corresponding INT2 and INT4 speedups are 2.67× and 2.30× over FlashInfer BF16, 1.48× and 1.28× over TensorRT-LLM FP8, and 1.16× and 1.00× over TensorRT-LLM NVFP4. We integrate FOSCAR into SGLang to benchmark serving efficiency. On long-context serving workloads, FOSCAR increases peak aggregate throughput by up to 170% and 341% over 2-bit and 16-bit baselines, respectively. We open-source FOSCAR as a kernel library at https://github.com/FutureMLS-Lab/FOSCAR.

1 INTRODUCTION

Autoregressive language-model decoding reads each active request's key–value (KV) cache on every decode step. The cache footprint grows with batch size and context length; storing it consumes high-bandwidth memory (HBM) capacity, and streaming it from HBM often dominates decode-attention latency. Low-bit KV quantization reduces that traffic directly. KIVI (Liu et al., 2024), Kitty (Xia et al., 2025), OSCAR (Zhou et al., 2026), KVQuant (Hooper et al., 2024), and QServe (Lin et al., 2025) store each cached value in 2–4 bits with a group-shared scale and offset.

Compressing the cache in memory does not eliminate on-chip operand preparation. KIVI (Liu et al., 2024), QServe (Lin et al., 2025), TurboAttention (Kang et al., 2025), BitDecoding (Du et al., 2026), SAW-INT4 (Jia et al., 2026), and OSCAR (Zhou et al., 2026) all convert or reconstruct low-bit values before each matrix product. This preparation consumes instruction-issue throughput, registers, and shared-memory bandwidth even when fused into attention without writing dequantized K and V to HBM. *Although bit-packing reduces HBM traffic, per-element operand preparation can still limit the resulting latency speedup.*

FOSCAR applies affine folding to the query–key (QK) and probability–value (PV) matrix products, which reduce over channels and tokens. For token-wise K, the kernel scales each group's partial QK result and adds the group's offset times the matching query sum. For token-wise V, the kernel scales the attention weights before PV and adds a weighted sum of offsets to the output. Channel-wise K instead scales the query before QK, while channel-wise V scales each group's partial PV result, and both add the corresponding offset-weighted sum. These per-group operations replace element-wise scale multiplication and offset addition, while bit manipulation constructs Tensor Core KV operands directly from packed integers. In exact arithmetic, affine folding produces the same attention output as dequantizing every KV value before the matrix products.

This paper makes four contributions:

[1]Together AI [2]University of Sydney. Correspondence to: Donglin Zhuang <donglinzhuang@outlook.com>.

Preprint.

- **Identify decode attention performance bottleneck.** We compare per-byte reconstruction instruction demand against peak instruction-issue and HBM throughputs on H100 and B200, showing how per-element conversion and affine arithmetic limit low-bit decode speedups (Sec. 3).
- **Efficient Tensor Core operand preparation via affine folding.** We formulate affine folding for independent token-wise or channel-wise K and V quantization, paired with bit-level Tensor Core operand construction, to feed Tensor Cores efficiently with lower instruction demand (Secs. 4.1 and 4.3).
- **Fast GPU kernels for Hopper and Blackwell.** We design warp-specialized decode pipelines for H100 register accumulators and B200 tensor-memory (TMEM) accumulators (Sec. 4.4).
- **Kernel and serving-engine evaluation.** We evaluate FOSCAR in standalone kernel benchmarks and integrated into SGLang (Zheng et al., 2024). FOSCAR INT2 achieves geometric-mean kernel-latency speedups over FlashInfer BF16 of 3.36× on H100 and 2.67× on B200. For long-context serving workloads, FOSCAR INT2 increases peak aggregate serving throughput by up to 170% and 341% over 2-bit and 16-bit baselines, respectively (Sec. 6).

2 Background and Motivation

2.1 Decode attention with a quantized KV cache

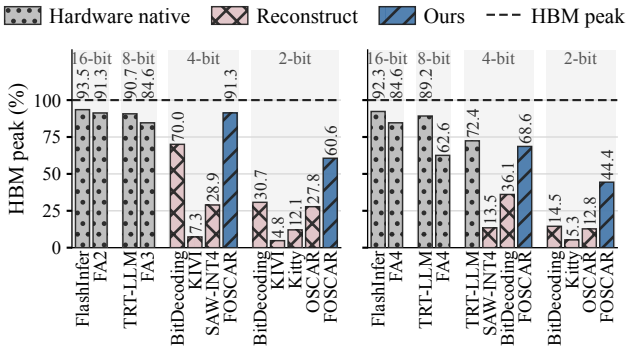
Each decode step streams the cached K and V tensors from HBM, so low-bit storage reduces the bytes that attention must read. Asymmetric quantization stores each cached element as a b-bit quantized value $q \in \{0, \ldots, 2^b - 1\}$. Each quantization group shares a scale s and an additive offset z to reconstruct an element x as

$$x = sq + z. \tag{1}$$

For either K or V, a token-wise group shares a scale and offset across channels within one token (Yao et al., 2022), whereas a channel-wise group shares them across tokens within one channel (Banner et al., 2019).

2.2 Limited latency savings in low-bit decode

Low-bit storage reduces the bytes that an attention kernel reads from HBM, but existing kernels still convert and dequantize each cached value before the matrix product. In Fig. 1, two- and four-bit kernels achieve a much smaller fraction of peak HBM throughput on their KV traffic than BF16 kernels, whether they fuse dequantization into attention (Lin et al., 2025; Kang et al., 2025; Du et al., 2026; Jia et al., 2026; Zhou et al., 2026) or into separate matrix-vector kernels (Liu et al., 2024).



(a) H100 (3.35 TB/s peak). (b) B200 (8.00 TB/s peak).

Figure 1. Effective KV access throughput of baseline kernels and FOSCAR at $B = 128$ and $KV = 32$K as a percentage of peak HBM throughput.

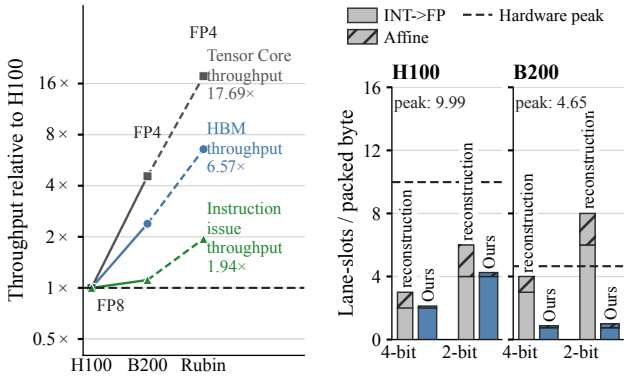
Unpacking each integer value and applying its group scale and offset consume instruction-issue slots. As the stored bit width drops, HBM transfer time falls while this per-value instruction cost stays fixed, shifting the bottleneck from memory bandwidth to instruction issue. Sec. 3 quantifies this instruction-issue cost and shows how affine folding removes per-value affine arithmetic.

3 Reconstruction Limits Low-Bit Decode Attention Speedups

A low-bit KV cache reduces HBM traffic, but the kernel must still construct floating-point operands fast enough to supply the Tensor Cores. In existing baselines, reconstruction involves converting integer values into floating-point operands, applying the affine transformation $sq + z$, and optionally performing one additional conversion to match the Tensor Core operand format (e.g., BF16 to FP8).

3.1 Low-bit baselines fall short of peak HBM throughput

We benchmark FlashInfer (Ye et al., 2025), TensorRT-LLM (NVIDIA Corporation, 2026e), FlashAttention-2 (Dao, 2024), FlashAttention-3 (Shah et al., 2024), FlashAttention-4 (Zadouri et al., 2026), BitDecoding (Du et al., 2026), KIVI (Liu et al., 2024), Kitty (Xia et al., 2025), SAW-INT4 (Jia et al., 2026), and OSCAR (Zhou et al., 2026) across KV-cache storage formats. We compute effective KV access throughput by dividing the bytes in one read of the stored values, scales, and offsets by decode time. Fig. 1 expresses this rate as a fraction of each GPU's peak HBM throughput at batch size $B = 128$ and context length $KV = 32$K.



(a) Peak throughput across GPU generations, normalized to H100 (labels show Rubin).

(b) Instruction-issue demand per packed KV byte on H100 and B200. INT→FP for converting packed integers to floating-point values; Affine is for apply scale and offset.

Figure 2. GPU throughput scaling and per-byte instruction-issue demand for operand construction and affine evaluation (FOSCAR shown at four query heads per KV head and group size 128).

FlashInfer BF16 exceeds 90% of peak on both H100 and B200. On B200, hardware-native TensorRT-LLM NVFP4 reaches 72.4% of peak in the 4-bit group. Among integer 4-bit baselines, BitDecoding is highest at 70.0% on H100 and 36.1% on B200, while its INT2 kernel reaches only 30.7% and 14.5%, respectively. FOSCAR reaches 91.3% and 60.6% at INT4 and INT2 on H100, and 68.6% and 44.4% on B200. Although low-bit baselines read fewer KV-cache bytes, their effective KV access throughput remains far below peak because kernel latency is bounded by the slowest pipeline stage. Sec. 3.2 shows how operand reconstruction can demand more instruction-issue throughput than the GPU provides at peak HBM throughput.

3.2 Reconstruction consumes instruction-issue slots

HBM transfers, operand preparation, softmax, and Tensor Core products all require instructions from the warp schedulers.

From H100 to B200, peak HBM throughput grows 2.39×, while peak instruction-issue throughput grows only 11% (NVIDIA Corporation, 2025b;a;c; 2026a). Peak dense Tensor Core throughput grows 4.55× when comparing H100 FP8 with B200 FP4 (NVIDIA Corporation, 2025b;a;c). Fig. 2a compares these peak throughputs across GPU generations[1].

Narrower stored values reduce HBM traffic without proportionally reducing reconstruction work. We count the instructions to convert packed integers to BF16, evaluate $sq + z$ in BF16, and convert the results to FP8 or FP4 when the Tensor Core instructions require those formats. One lane-slot corresponds to one instruction issued for one warp lane (NVIDIA Corporation, 2026b). Let a_{issue} denote the lane-slots required to construct an operand and evaluate $sq + z$ for one quantized value, C_{issue} the GPU's peak instruction-issue throughput in lane-slots/s, and B_{HBM} its peak HBM throughput in bytes/s. Each quantized value occupies $B_{\text{elem}} = 0.5$ bytes in INT4 and 0.25 bytes in INT2, excluding scales and offsets. Streaming the cache at peak HBM throughput remains within peak instruction-issue throughput only when

$$\underbrace{\frac{a_{\text{issue}}}{B_{\text{elem}}}}_{\text{issue demand per byte}} \leq \underbrace{\frac{C_{\text{issue}}}{B_{\text{HBM}}}}_{\text{peak issue per byte}} \tag{2}$$

Fig. 2b compares this per-byte issue demand against the hardware peak from Eq. (2) on H100 and B200.

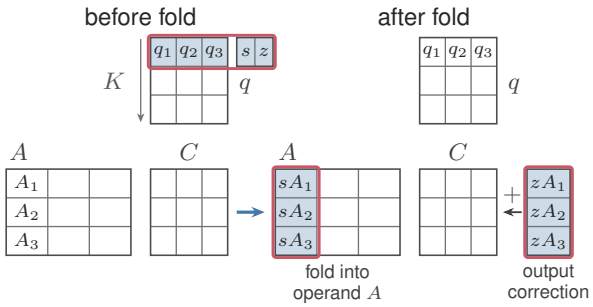
Preparing four BF16 Tensor Core operands on H100 requires two `SHF` and two `LOP3` instructions for INT→FP conversion. Two BF16 `HFMA2` instructions then evaluate the affine transformation $sq + z$, each producing two BF16 values in a 32-bit register (Frantar et al., 2025). The total is six instructions, or six lane-slots per warp lane. Four values occupy two bytes in INT4 or one byte in INT2, giving $6/2 = 3$ or $6/1 = 6$ lane-slots/byte in Fig. 2b, respectively. Therefore, reconstruction alone can consume 30% and 60% of H100's peak hardware budget of 9.99 lane-slots/byte, leaving little room for other parts of the kernel.

On B200, two `F2FP` instructions convert the four BF16 values directly to FP8 or FP4 and pack them into registers.[2] Reconstructing four values therefore takes $6 + 2 = 8$ lane-slots, or $8/2 = 4$ lane-slots/byte for INT4 and $8/1 = 8$ lane-slots/byte for INT2. At peak HBM throughput, INT2 reconstruction *exceeds* B200's hardware peak of 4.65 lane-slots/byte, and INT4 reconstruction alone consumes 86% of that peak.

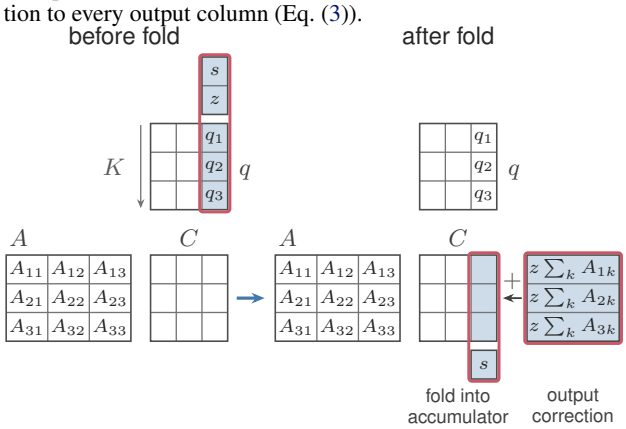
FOSCAR removes per-value affine evaluation and never materializes a reconstructed K or V tile. Instead, it constructs Tensor Core operands directly from q, applies each group's scale to the other matrix input or partial product, and adds the offset as an output correction. At four query heads per KV head and group size 128, affine folding reduces issue demand in Fig. 2b to 2.13 and 4.25 lane-slots/byte (INT4 and INT2) on H100, and to 0.88 and 1.00 on B200.

[1] Rubin's figures are preliminary; we infer its instruction-issue throughput from FP32 CUDA Core throughput assuming 128 lane-slots per SM cycle (NVIDIA Corporation, 2023; 2026d;a).

[2] Before PTX ISA 9.1 (NVIDIA Corporation, 2026c), the conversion uses FP32 intermediates and takes six instructions per four values with CUDA 13.0. Reconstruction then costs twelve instructions, or 6 and 12 lane-slots/byte for INT4 and INT2, respectively.



(a) Operand-fold scales a column of A and adds the offset correction to every output column (Eq. (3)).



(b) Acc-fold scales a partial output column and adds the offset-weighted row sums of A (Eq. (4)).

Figure 3. Affine folding of $C = A(sq + z)$, where K is the reduction axis.

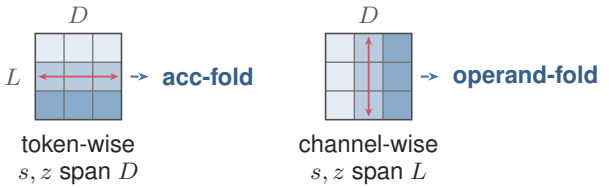
4 FOSCAR DESIGN

FOSCAR consists of three components:

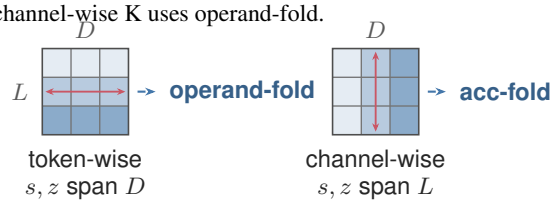
- *Affine folding* moves shared scales and offsets from individual KV values to matrix-product inputs and results so that Tensor Cores consume operands built directly from packed quantized values, reducing the affine demand in Fig. 2b (Sec. 4.1).
- *Bit-level Tensor Core operand construction* builds Tensor Core operands directly from packed quantized values, reducing the INT→FP conversion demand in Fig. 2b (Sec. 4.3).
- *Warp-specialized GPU pipelines* overlap data movement, operand preparation, matrix products, and affine corrections on Hopper and Blackwell (Sec. 4.4).

4.1 The affine folding mechanism

Consider $C = A(sq + z)$, where $A \in \mathbb{R}^{M\times K}$, $q \in \mathbb{R}^{K\times N}$ holds quantized values, and k indexes the reduction. When each row k of q shares a scale s_k and offset z_k across columns j, distributing reconstruction through the product



(a) $QK^\top$ reduces over D, so token-wise K uses acc-fold and channel-wise K uses operand-fold.



(b) PV reduces over L, so token-wise V uses operand-fold and channel-wise V uses acc-fold.

Figure 4. Affine folding modes for token-wise and channel-wise K and V tiles.

gives

$$C_{i,j} = \sum_k A_{i,k}(s_k q_{k,j} + z_k) = \underbrace{\sum_k (s_k A_{i,k}) q_{k,j}}_{\text{scaled input}} + \underbrace{\sum_k z_k A_{i,k}}_{\text{output correction}} . \tag{3}$$

We call this *operand-fold* (Fig. 3a): the scale multiplies the other input A, and the offset contributes a separate reduction of A without reconstructing individual entries of q.

When each column j of q shares a scale s_j and offset z_j across the reduction index k, both terms factor out of the sum:

$$C_{i,j} = \sum_k A_{i,k}(s_j q_{k,j} + z_j) = \underbrace{s_j \sum_k A_{i,k} q_{k,j}}_{\text{scaled partial result}} + \underbrace{z_j \sum_k A_{i,k}}_{\text{output correction}} . \tag{4}$$

We call this *acc-fold* (Fig. 3b): the scale multiplies the accumulated product, and the offset multiplies the corresponding row sum of A. Both *operand-fold* and *acc-fold* preserve the result of applying $sq + z$ element-wise in exact arithmetic.

In attention, K and V arrive as $[L, D]$ tiles, with L cached tokens and head dimension D. Token-wise groups share a scale and offset along D within each token, whereas channel-wise groups share them along L within each channel. Because $QK^\top$ reduces over D, token-wise K uses acc-fold and channel-wise K uses operand-fold into Q (Fig. 4a). Because PV reduces over L, token-wise V uses operand-fold into P and channel-wise V uses acc-fold (Fig. 4b). FOSCAR allows K and V to select their quantization and folding schemes independently according to the cache's quantization policy. Fig. 5 illustrates where scales and offsets are

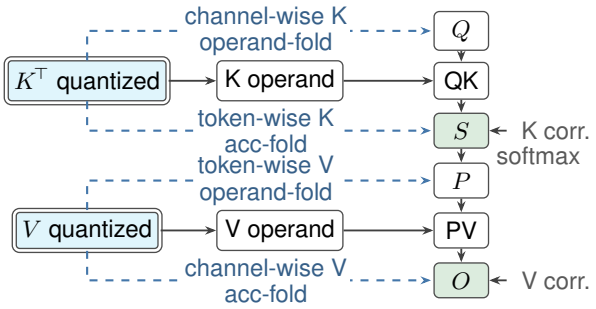


Figure 5. Affine folding in attention. Dashed arrows show where each K and V quantization scheme applies its scales.

applied in the attention algorithm. FOSCAR retains online softmax (Dao et al., 2023; 2022; Dao, 2024) and changes only the QK and PV operand paths and their output corrections.

With h_q query heads per KV head and group size g, affine folding reduces scalar scale-and-offset operations from one multiply-add per stored value to $2h_q/g$ (Appendix C).

4.2 Affine folding spans multiple quantization groups

Under acc-fold, an unscaled partial product cannot cross a quantization-group boundary. When a reduction spans multiple groups, the kernel applies each group's scale and offset to its partial product before adding the result to the running accumulator (Eq. (4)). We call this partial product *group-local* because it sums only over the values that share one scale and offset. Operand-fold instead scales the other matrix input before multiplication, so the accumulator can span multiple groups without rescaling; its offset correction still follows Eq. (3).

Both the Hopper and Blackwell kernels enforce the same ordering constraints on these corrections, although they assign the work to different warpgroups (Sec. 4.4). For operand-fold, the kernel can accumulate offset contributions for the same output separately and add their sum later. QK corrections must precede softmax, while PV corrections must precede division by the accumulated softmax denominator.

4.3 Constructing Tensor Core operands from packed integers

FOSCAR constructs floating-point Tensor Core operands directly from packed integer bit patterns.[3] Hopper combines each integer value with a fixed BF16 bit pattern, a technique known as mantissa injection (Kim et al., 2022; Frantar et al., 2025; Du et al., 2026). Combining $q \in \{0, \ldots, 15\}$ with `0x4300` via bitwise OR produces the exact BF16 value $y = q + 128$.

[3]The Hopper kernel uses BF16 operands with `wgmma`. The Blackwell kernel uses `tcgen05` with E4M3 inputs for Q and P, E4M3 KV operands for INT4, and E2M1 KV operands for INT2 (using the `kind::f8f6f4` instruction mode (NVIDIA Corporation, 2026c)).

Mapping	$q \to y$	y	(α, β)	ALU/elem.
Hopper				
INT4→BF16	`0x4300` | q	$q + 128$	$(1, 128)$	1.00
INT2→BF16	`0x4300` | q	$q + 128$	$(1, 128)$	1.00
Blackwell				
INT4→E4M3	q	$q \cdot 2^{-9}$	$(2^{-9}, 0)$	0.38
INT2→E2M1	q	$q/2$	$(1/2, 0)$	0.19

Table 1. Tensor Core operand mappings and their static ALU costs per stored value. The $q \to y$ column gives the bit-level construction of y from q.

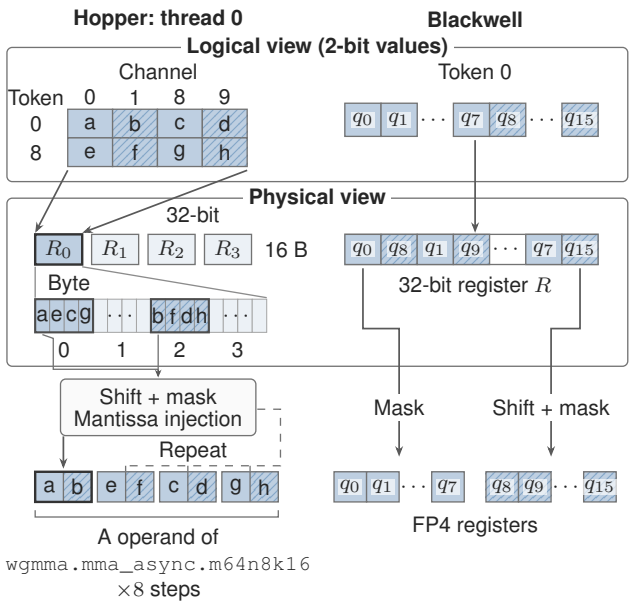
On Blackwell, FOSCAR can convert integer values into floating point more efficiently by *directly placing each integer's bit pattern in the low bits of a floating-point operand and filling the remaining bits with zeros*. For example, the bit patterns of the INT2 values $0, 1, 2, 3$ represent $0, 0.5, 1, 1.5$ in FP4 (E2M1), giving $y = q/2$. INT4 similarly yields FP8 (E4M3) operands with $y = q \cdot 2^{-9}$ (Lin, 2025).

These constructions produce operands of the form $y = \alpha q + \beta$ ($\alpha \neq 0$) whose numerical values differ from the stored integers. To account for (α, β) without extra per-element instructions, we fold both constants into each group's shared scale and offset. Substituting $q = (y - \beta)/\alpha$ into $x = sq + z$ (Eq. (1)) gives

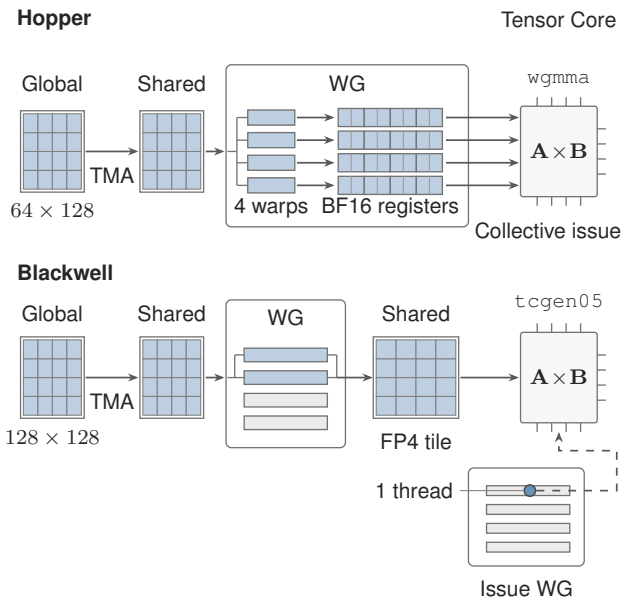
$$x = s'y + z', \qquad s' = s/\alpha, \qquad z' = z - \beta s'. \tag{5}$$

Affine folding then applies s' and z' in place of s and z, leaving the attention result unchanged in exact arithmetic (Sec. 4.1). s' and z' are precomputed in FP32 when the quantized KV cache is stored in HBM. Table 1 summarizes the $q \to y$ mappings, the (α, β) pairs used for compensation, and the ALU cost of converting each integer KV element into a Tensor Core operand.

Fig. 6a shows how FOSCAR packs INT2 K values in each 32-bit register to match these bit-level constructions. On Hopper, each byte interleaves pairs of 2-bit values so that one bit shift and one masked merge (`LOP3`) produce two BF16 operands in a 32-bit register. On Blackwell, each 32-bit register holds 16 two-bit values arranged as eight channel pairs; one bit shift and two masks unpack the register directly into two 32-bit registers holding 16 FP4 operands. Both kernels use the Tensor Memory Accelerator (TMA) to copy packed tiles from global to shared memory (Fig. 6b). Hopper consumers build K operands in registers and collectively issue `wgmma`; Blackwell prepares a shared-memory operand tile that a separate issue thread supplies to Tensor Cores by issuing `tcgen05` (NVIDIA Corporation, 2026c). Appendix A extends the INT2 K example to the packing



(a) Packed register layout and bit-level operand construction.



(b) K operand flow from global memory to the Tensor Core. Tile shapes are logical (tokens × channels).

Figure 6. Register packing and global-to-Tensor-Core dataflow for INT2 K in FOSCAR.

and dataflow of INT2 V and INT4 KV operands on both GPUs.

4.4 Warp-specialized GPU pipelines

We design separate warp-specialized pipelines for Hopper and Blackwell. Hopper's `wgmma` accumulates into registers owned by its issuing warpgroup, whereas Blackwell's `tcgen05` accumulates into tensor memory (TMEM) accessible to other warpgroups in the CTA (NVIDIA Corporation, 2026c).

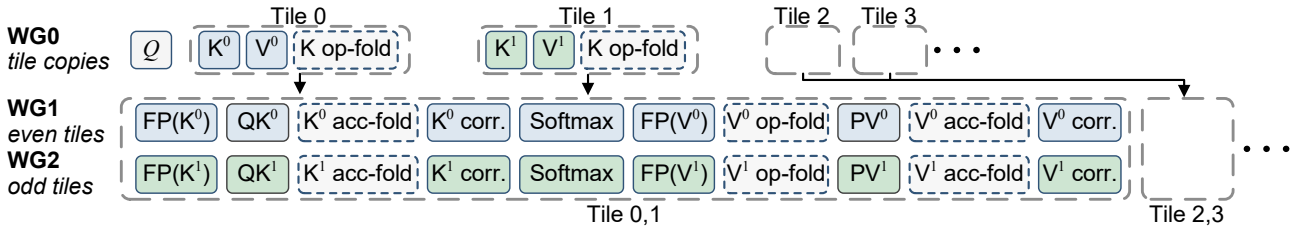
On Hopper, one producer warpgroup issues TMA copies, and two consumer warpgroups process alternating even and odd KV tiles (Fig. 7a). Each consumer builds KV operands and runs its own QK–softmax–PV chain, keeping online-softmax state and output accumulators in registers. Each consumer applies scales and offset corrections according to Fig. 5: K corrections precede softmax, and V corrections update the output as described in Sec. 4.2. For channel-wise K, the producer warpgroup prepares the scaled Q inputs. At the end of the sequence, one consumer merges the two warpgroups' softmax states and partial outputs into the final CTA output.

Blackwell assigns copy and matrix-product issue, operand preparation, softmax, and output accumulation to separate warpgroups (Fig. 7b). The issue warpgroup starts TMA transfers and `tcgen05` products, while the preparation warpgroup constructs KV operands in shared memory. The softmax warpgroup reads QK accumulators from TMEM, applies K offset corrections (and K scales for token-wise K) in FP32, computes the softmax weights P, and scales P for token-wise V (quantizing folded Q and P tiles to E4M3 with a per-row scale). The output warpgroup reads PV accumulators from TMEM, maintains the running output, and applies V offset corrections (and V scales for channel-wise V); for channel-wise K, it also prepares the scaled Q inputs. Shared-memory buffers pass operands between warpgroups, while TMEM passes matrix-product accumulators.

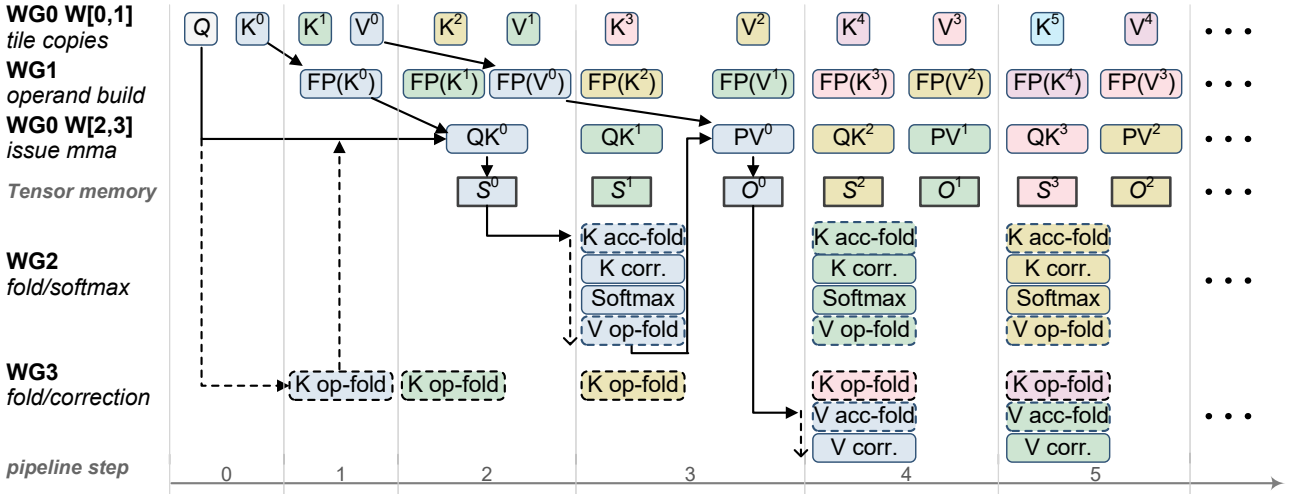
Multi-stage buffers in shared memory and TMEM allow producer warpgroups to work on later tiles while downstream warpgroups process earlier tiles. We tune the buffer depth and producer lead distance empirically for each workload; for clarity, Fig. 7b illustrates a one-step lead.

5 Serving Engine Integration

We integrate FOSCAR into SGLang as a backend for paged decode attention (Zheng et al., 2024). FOSCAR incorporates common optimization techniques, including split-KV, which divides long contexts across CTAs to increase parallelism. To better understand how FOSCAR benefits existing serving systems with quantized KV caches, we integrate FOSCAR with OSCAR (Zhou et al., 2026) as a drop-in replacement for its INT2 kernel and keep OSCAR's quantization policy, rotations, and BF16 windows (Sec. 6.4) unchanged.



(a) H100: `WG0` issues the tile copies using TMA, and two consumer warpgroups alternate even and odd tiles.



(b) B200: `WG0` issues the tile copies and `tcgen05` matrix instructions, while `WG1`–`WG3` prepare KV operands, apply affine folding, and compute softmax on CUDA Cores.

Figure 7. FOSCAR's Hopper and Blackwell pipelines. $\mathrm{FP}(X)$ constructs X's floating-point Tensor Core operand from packed integers (Sec. 4.3). `corr.` denotes the output correction, and `WG`x `W[`y`]` denotes warpgroup x, warp y. Dashed boxes execute only for the corresponding K or V quantization axis.

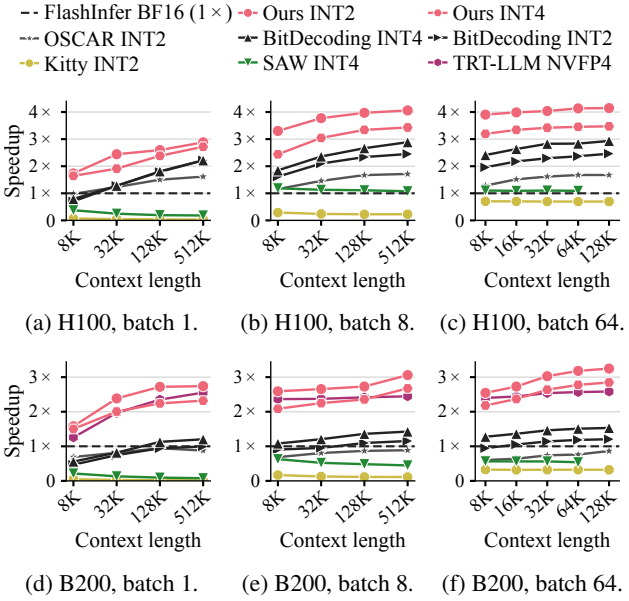
6 Evaluation

We implement FOSCAR in CuTe DSL (CUTLASS 4.8, CUDA 13.0) and evaluate it on NVIDIA H100 (80 GB HBM3) and B200 (180 GB HBM3e) GPUs with $H_Q = 32$, $H_{KV} = 8$, $D = 128$, page size 64, and token-wise K and V quantization groups of $g = 128$ values with FP32 scales and offsets. Token-wise K and V exercise both acc-fold on QK and operand-fold on PV; Appendix B evaluates all four combinations of token-wise and channel-wise K and V, where FOSCAR achieves 1.75–4.00× speedups over FlashInfer BF16 on H100 and 1.50–2.74× on B200. We evaluate batch sizes 1 and 8 at context lengths of 8K, 32K, 128K, and 512K, and batch size 64 at 8K, 16K, 32K, 64K, and 128K. Each measurement takes the median of five timed iterations (excluding an L2 flush before each call) after five warmups, and includes all decode kernels and split reduction. Plotted speedups normalize each kernel's latency to FlashInfer BF16, whereas speedups quoted in the text compare FOSCAR directly against each named baseline within the same benchmark series (using five-run medians for B200 INT4 vendor baselines and three-run medians for BitDecoding and SAW).

6.1 Kernel performance

Figs. 8 and 9 compare FOSCAR against sub-byte and wider-format baselines. On H100, FOSCAR INT2 achieves a 3.36× geometric-mean speedup over FlashInfer BF16 and 2.03× over TensorRT-LLM FP8, with maxima of 4.15× and 2.44×. On B200, the geometric-mean speedups are 2.67× over BF16, 1.48× over TensorRT-LLM FP8, and 1.16× over TensorRT-LLM NVFP4, with maxima of 3.25×, 1.64×, and 1.26×. These vendor kernels are invoked through FlashInfer, and FOSCAR INT2 outperforms them across all tested batch sizes and context lengths. Against OSCAR INT2 and BitDecoding INT2, FOSCAR INT2 achieves geometric-mean speedups of 2.31× and 1.77× on H100, and 3.44× and 2.78× on B200.

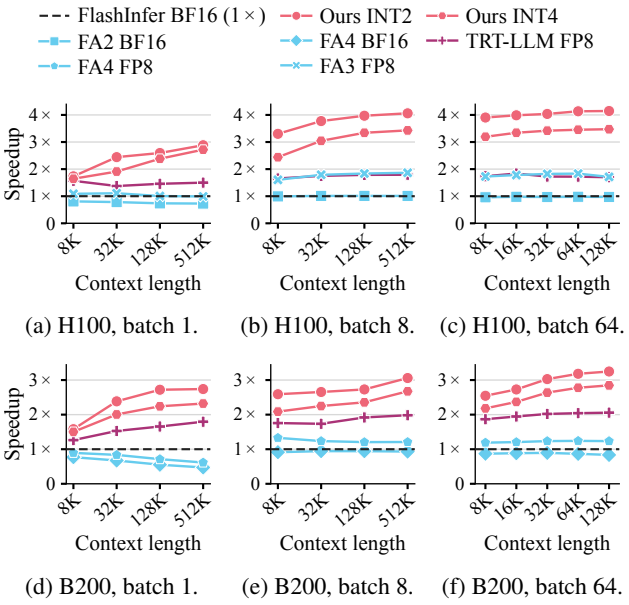
On H100, FOSCAR INT4 achieves geometric-mean speedups of 2.83× over FlashInfer BF16, 1.71× over TensorRT-LLM FP8, 1.32× over BitDecoding INT4, and 4.19× over SAW INT4. On B200, FOSCAR INT4 achieves geometric-mean speedups of 2.30× over FlashInfer BF16, 1.28× over TensorRT-LLM FP8, 1.94× over BitDecoding INT4, and 6.93× over SAW INT4. It matches na-



(a) H100, batch 1. (b) H100, batch 8. (c) H100, batch 64.

(d) B200, batch 1. (e) B200, batch 8. (f) B200, batch 64.

Figure 8. Kernel speedups of FOSCAR and 2- and 4-bit baselines over FlashInfer BF16.

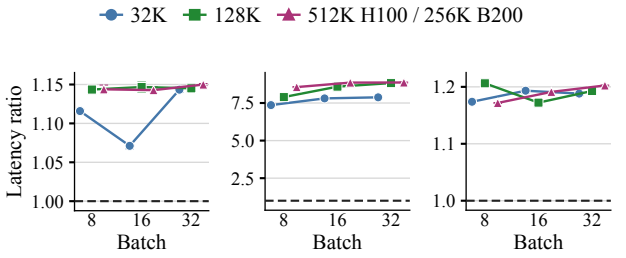


(a) H100, batch 1. (b) H100, batch 8. (c) H100, batch 64.

(d) B200, batch 1. (e) B200, batch 8. (f) B200, batch 64.

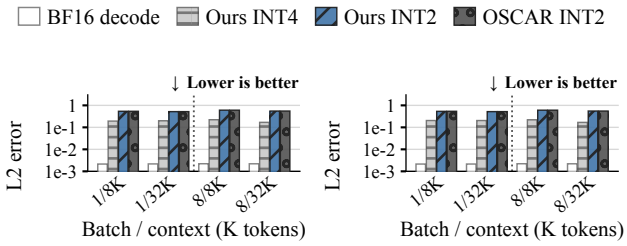
Figure 9. Kernel speedups of FOSCAR and 8- and 16-bit baselines over FlashInfer BF16. The dashed line marks parity.

tive NVFP4 overall (1.00×, ranging from 0.88× to 1.20×; faster at 32K–128K for batch 64), while returning BF16 output instead of NVFP4's FP8 output. Appendix C further evaluates how FOSCAR INT2 speedup scales across $H_{KV} \in [1, 32]$.



(a) H100: reconstruction versus folding. (b) B200: E4M3 reconstruction versus folding. (c) B200: folded E4M3 versus E2M1.

Figure 10. FOSCAR INT2 kernel latency ratios between ablation variants at equal split-KV counts (first variant's latency divided by the second's). Panels (b) and (c) decode an INT2 cache into E4M3 or E2M1 Tensor Core operands on B200; error bars show paired 95% confidence intervals across ten runs.



(a) H100. (b) B200.

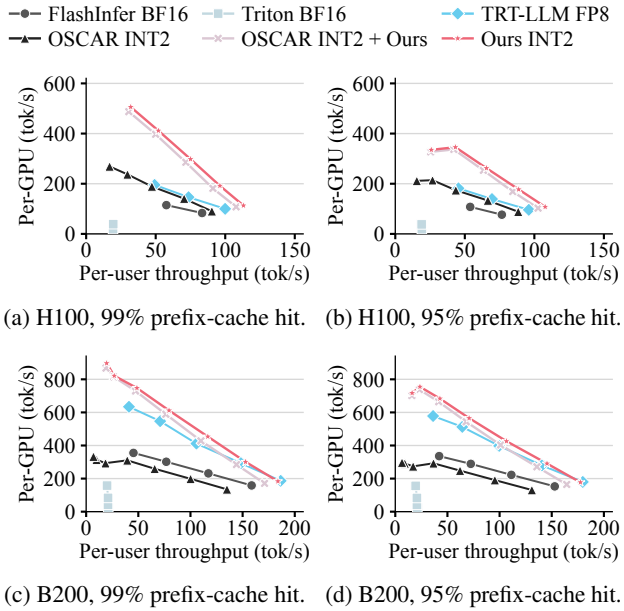
Figure 11. L_2 output error of BF16 decode (FlashInfer), FOSCAR INT4, FOSCAR INT2, and OSCAR INT2 on the synthetic input (logarithmic scale).

6.2 Kernel accuracy

Fig. 11 compares the L_2 output error of BF16 decode (FlashInfer), FOSCAR INT4, FOSCAR INT2, and OSCAR INT2. To stress-test quantization, our synthetic input amplifies eight K channels, ramps K's per-token scale along the sequence, and rescales K so that pre-softmax attention logits have unit standard deviation. For INT2, FOSCAR and OSCAR decode the same quantized cache written by OSCAR's clipped quantizer without BF16 windows. Both INT2 kernels yield L_2 errors of 0.52–0.61, compared with 0.16–0.22 for FOSCAR INT4 and $(2.13\text{–}2.25) \times 10^{-3}$ for BF16 decode. Under unclipped min–max INT2 quantization, the L_2 error rises to 0.86–1.03. Compared directly against the dequantized INT2 cache, FOSCAR's kernel arithmetic error is only $(1.68\text{–}1.76) \times 10^{-3}$ on both H100 and B200, matching BF16 decode and confirming that the 0.52–0.61 error stems from 2-bit quantization rather than affine folding.

6.3 Kernel ablation

On INT2 caches with equal split-KV counts within each pair, Fig. 10 ablates affine folding against per-value reconstruction on H100 and B200, and FP4 (E2M1) against FP8



(a) H100, 99% prefix-cache hit. (b) H100, 95% prefix-cache hit.

(c) B200, 99% prefix-cache hit. (d) B200, 95% prefix-cache hit.

Figure 12. Per-user and aggregate Qwen3-8B serving throughput on a single GPU with 100K input and 1K output tokens.

(E4M3) KV Tensor Core operands on B200. On H100, FOSCAR's speedup over external baselines combines its warp-specialized kernel design with affine folding, which adds a 1.07–1.15× speedup over per-value reconstruction (Fig. 10a) as expected when H100 reconstruction remains within the hardware issue peak (Fig. 2b). On B200, the reconstruction baseline follows the standard approach of dequantizing INT2 values into FP8 (E4M3) Tensor Core operands before each matrix product; because this per-value reconstruction exceeds B200's issue peak, affine folding achieves a 7.36–8.86× speedup (Fig. 10b). Switching the folded B200 INT2 kernel from FP8 (E4M3) to FP4 (E2M1) KV operands and their matching packing layout provides an additional 1.17–1.21× speedup (Fig. 10c).

6.4 Serving throughput

We serve Qwen3-8B with 100K input tokens, 1K output tokens, and 99% or 95% prefix-cache hits (one cached prefix per request), sharing a BF16 prefill backend across configurations. We evaluate two INT2 configurations to show both realistic accuracy-preserving integration and pure 2-bit kernel speed: *OSCAR INT2 + Ours* replaces OSCAR's INT2 decode kernel with FOSCAR while keeping OSCAR's rotations and BF16 windows (128 initial and 512 recent tokens, $< 0.7\%$ of the 100K context), whereas *Ours INT2* uses FOSCAR's token-wise INT2 K and V cache without recent windows or rotations. We vary concurrency in powers of two up to 16 on H100 and 32 (plus 48) on B200 to measure per-user (median request output tokens/s) and aggregate (total output tokens/s) throughput (Fig. 12).

Table 2. Qwen3-8B serving accuracy on H100 (mean ± sample standard deviation across five runs).

Configuration	GPQA-Diamond (%)	GSM8K (%)
FlashInfer BF16	56.36 ± 1.84	95.71 ± 0.11
OSCAR INT2	54.65 ± 2.18	95.74 ± 0.38
OSCAR INT2 + Ours	55.45 ± 1.49	95.92 ± 0.15

At a 99% prefix-cache hit rate, *OSCAR INT2 + Ours* reaches a peak aggregate throughput of 487 tokens/s on H100 and 868 tokens/s on B200, compared with 269 and 333 tokens/s for OSCAR (81% and 161% higher). *Ours INT2* reaches 506 tokens/s on H100 and 897 tokens/s on B200, improving over OSCAR by 88% and 170%; this also exceeds FlashInfer BF16's peaks of 115 tokens/s on H100 (at concurrency 2) and 355 tokens/s on B200 (at concurrency 8) by 341% and 153%. At 95% prefix hits, *OSCAR INT2 + Ours* and *Ours INT2* reach 337 and 346 tokens/s versus OSCAR's 214 tokens/s on H100, and 738 and 755 tokens/s versus 297 tokens/s on B200. Across cache formats, peak throughput reflects both decoder speed and the number of concurrent requests that fit in memory.

6.5 Serving accuracy

Table 2 compares FlashInfer BF16, OSCAR INT2, and *OSCAR INT2 + Ours* on GPQA-Diamond (Rein et al., 2023) and GSM8K on H100, reporting the mean and sample standard deviation across five runs per configuration. Every question-paired 95% confidence interval for the accuracy difference between configurations spans zero, so five runs show no measurable accuracy gain or loss (though five runs do not establish formal equivalence). Median prompt lengths are 243 tokens for GPQA-Diamond and 91 tokens for GSM8K; because *OSCAR INT2 + Ours* decodes the identical quantized cache as OSCAR with $< 1.8 \times 10^{-3}$ arithmetic error (Sec. 6.2), it preserves OSCAR's cache accuracy while accelerating decode.

7 Related Work

7.1 KV-cache quantization algorithms

KIVI (Liu et al., 2024), KVQuant (Hooper et al., 2024), and Kitty (Xia et al., 2025) establish low-bit KV formats and motivate channel-wise grouping for K and token-wise grouping for V. Atom (Zhao et al., 2024) and QServe (Lin et al., 2025) co-design 4-bit quantization formats and serving kernels, while GEAR (Kang et al., 2024) adds low-rank and sparse error corrections. FOSCAR retains uniform affine quantization and changes how a Tensor Core kernel evaluates the quantized cache. Nonlinear lookup formats and sparse outlier representations cannot be expressed as a single affine transform and require separate kernel support.

7.2 Quantized attention kernels

BitDecoding provides low-bit Tensor Core attention kernels across several GPU generations (Du et al., 2026). On Hopper, it converts stored integer values and applies scales and offsets before the matrix products. On Blackwell, it uses block-scaled FP4 instructions that restrict scales to the contraction axis, omit offsets, and quantize Q and each P tile to FP4. TensorRT-LLM's XQA kernel folds a scalar INT8 or FP8 KV scale into the QK and PV accumulators (NVIDIA Corporation, 2026e). QServe fuses low-bit KV dequantization into CUDA-core batched matrix-vector products (Lin et al., 2025). TurboAttention converts sub-8-bit values into INT8 operands for Tensor Core attention (Kang et al., 2025). The SageAttention kernels quantize full-precision Q, K, and V inputs for prefill and general attention workloads, including FP4 Tensor Core execution on Blackwell (Zhang et al., 2025c;a;b). SAW-INT4 (Jia et al., 2026) combines token-wise affine INT4 caches with a block-diagonal Hadamard rotation, while OSCAR (Zhou et al., 2026) combines a mixed INT2/BF16 cache with offline spectral rotations; both kernels reconstruct cached values before their matrix products. FOSCAR instead applies affine folding across independent token-wise or channel-wise K and V caches, folding group scales and non-zero offsets into the Q or P inputs or into group-local partial products while keeping Q and P in BF16 or FP8. Our serving evaluation combines affine folding with OSCAR's cache policy and rotations (Sec. 6.4).

7.3 Attention pipelines and kernel reparameterization

FlashAttention-2 introduces work partitioning for tiled attention (Dao, 2024), FlashAttention-3 overlaps producer and consumer warpgroups through warp specialization (Shah et al., 2024), FlashInfer provides paged-cache attention kernels for serving engines (Ye et al., 2025), and FlashAttention-4 studies how Blackwell's higher Tensor Core throughput shifts bottlenecks toward on-chip staging and non-matmul instructions (Zadouri et al., 2026). RepVGG (Ding et al., 2021) folds training-time convolution branches into inference kernels, CODA (Guo et al., 2026) rewrites Transformer operators into GEMM epilogues, and quantized GEMM kernels (Zhao et al., 2024; Lin et al., 2025) rescale group partial sums or subtract zero-point activation sums in their epilogues. FOSCAR combines warp-specialized attention pipelining with scale-and-offset folding across both QK and PV under online softmax, where the eliminated per-element work grows with context length.

7.4 Quantized inference architecture

Oaken combines a KV quantization policy with custom hardware for online and offline compression (Kim et al., 2025). Weight-only kernels such as Marlin (Frantar et al., 2025; Lin, 2025), FLUTE (Guo et al., 2024), and LUT-GEMM (Park et al., 2024) optimize single matrix products over quantized model weights. FOSCAR instead targets the QK and PV matrix products of decode attention, which are linked by online softmax, on unmodified Hopper and Blackwell Tensor Cores.

8 Conclusion

FOSCAR accelerates decode attention over low-bit KV caches through affine folding, which moves each quantization group's shared scale to a matrix-product input or partial result and applies its offset as an output correction. Its Hopper and Blackwell kernels construct Tensor Core KV operands from packed quantized values through bit manipulation, while affine folding preserves the attention output of the reconstructed cache in exact arithmetic. On H100 and B200, FOSCAR INT2 outperforms FlashInfer BF16 and TensorRT-LLM FP8 across all tested batch sizes and context lengths on the main benchmark grid. Qwen3-8B serving experiments also show throughput gains of up to 170% over baselines in settings with long contexts and high cache hit rates that align with popular agentic workloads.

References

Banner, R., Nahshan, Y., and Soudry, D. Post training 4-bit quantization of convolutional networks for rapid-deployment. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E. B., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 7948–7956, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/c0a62e133894cdce435bcb4a5df1db2d-Abstract.html.

Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=mZn2Xyh9Ec.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/

67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

Dao, T., Haziza, D., Massa, F., and Sizov, G. Flash-Decoding for long-context inference. PyTorch Blog, 2023. URL https://pytorch.org/blog/flash-decoding/. Accessed 2026-08-14.

Ding, X., Zhang, X., Ma, N., Han, J., Ding, G., and Sun, J. Repvgg: Making vgg-style convnets great again. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*, pp. 13733–13742. Computer Vision Foundation / IEEE, 2021. doi: 10.1109/CVPR46437.2021.01352. URL https://openaccess.thecvf.com/content/CVPR2021/html/Ding_RepVGG_Making_VGG-Style_ConvNets_Great_Again_CVPR_2021_paper.html.

Du, D., Cao, S., Cheng, J., Mai, L., Cao, T., and Yang, M. Bitdecoding: Unlocking tensor cores for long-context llms with low-bit KV cache. In *IEEE International Symposium on High Performance Computer Architecture, HPCA 2026, Sydney, Australia, January 31 - Feb. 4, 2026*, pp. 1–13. IEEE, 2026. doi: 10.1109/HPCA68181.2026.11408481. URL https://doi.org/10.1109/HPCA68181.2026.11408481.

Frantar, E., Castro, R. L., Chen, J., Hoefler, T., and Alistarh, D. MARLIN: mixed-precision auto-regressive parallel inference on large language models. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2025, Las Vegas, NV, USA, March 1-5, 2025*, pp. 239–251. ACM, 2025. doi: 10.1145/3710848.3710871. URL https://doi.org/10.1145/3710848.3710871.

Guo, H., Brandon, W., Cholakov, R., Ragan-Kelley, J., Xing, E. P., and Kim, Y. Fast matrix multiplications for lookup table-quantized llms. In Al-Onaizan, Y., Bansal, M., and Chen, Y. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, volume EMNLP 2024 of *Findings of ACL*, pp. 12419–12433. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.FINDINGS-EMNLP.724. URL https://doi.org/10.18653/v1/2024.findings-emnlp.724.

Guo, H., Zhang, J., Menon, A., Guessous, D., Thakkar, V., Kim, Y., and Dao, T. CODA: rewriting transformer blocks as gemm-epilogue programs. *CoRR*, abs/2605.19269, 2026. doi: 10.48550/ARXIV.2605.19269. URL https://doi.org/10.48550/arXiv.2605.19269.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. Kvquant: Towards 10 million context length LLM inference with KV cache quantization. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/028fcbcf85435d39a40c4d61b42c99a4-Abstract-Conference.html.

Jia, J., Li, J., Zhou, Z., Heo, J. H., Wang, J., Dao, T., Song, S. L., Athiwaratkun, B., Xu, C., Zhang, T., and Wu, X. SAW-INT4: system-aware 4-bit kv-cache quantization for real-world LLM serving. *CoRR*, abs/2604.19157, 2026. doi: 10.48550/ARXIV.2604.19157. URL https://doi.org/10.48550/arXiv.2604.19157.

Kang, H., Zhang, Q., Kundu, S., Jeong, G., Liu, Z., Krishna, T., and Zhao, T. GEAR: an efficient KV cache compression recipe for near-lossless generative inference of LLM. *CoRR*, abs/2403.05527, 2024. doi: 10.48550/ARXIV.2403.05527. URL https://doi.org/10.48550/arXiv.2403.05527.

Kang, H., Bharadwaj, S., Hensman, J., Krishna, T., Rühle, V., and Rajmohan, S. Turboattention: Efficient attention approximation for high throughputs llm. In Zaharia, M., Joshi, G., and Lin, Y. C. (eds.), *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025*. OpenReview.net/mlsys.org, 2025. URL https://openreview.net/forum?id=bPfpSO2zX4.

Kim, M., Hong, S., Ko, R., Choi, S., Lee, H., Kim, J., Kim, J., and Park, J. Oaken: Fast and efficient LLM serving with online-offline hybrid KV cache quantization. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture, ISCA 2025, Tokyo, Japan, June 21-25, 2025*, pp. 482–497. ACM, 2025. doi: 10.1145/3695053.3731019. URL https://doi.org/10.1145/3695053.3731019.

Kim, Y. J., Henry, R., Fahim, R., and Awadalla, H. H. Who says elephants can't run: Bringing large scale moe models into cloud scale production. *CoRR*, abs/2211.10017, 2022. doi: 10.48550/ARXIV.2211.10017. URL https://doi.org/10.48550/arXiv.2211.10017.

Lin, J. Add W4A8 support for the Marlin kernel. vLLM pull request #24722, 2025. URL https://github.com/vllm-project/vllm/pull/24722. Accessed 2026-08-19.

Lin, Y., Tang, H., Yang, S., Zhang, Z., Xiao, G., Gan, C., and Han, S. Qserve: W4A8KV4 quantization and system co-design for efficient LLM serving. In Zaharia,

M., Joshi, G., and Lin, Y. C. (eds.), *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025*. OpenReview.net/mlsys.org, 2025. URL https://openreview.net/forum?id=1FfmStySS1.

Liu, Z., Yuan, J., Jin, H., Zhong, S. H., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In Salakhutdinov, R., Kolter, Z., Heller, K. A., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, volume 235 of *Proceedings of Machine Learning Research*, pp. 32332–32344. PMLR / OpenReview.net, 2024. URL https://proceedings.mlr.press/v235/liu24bz.html.

NVIDIA Corporation. *GPU Performance Background User's Guide*. NVIDIA Corporation, 2023. URL https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html. Accessed 2026-08-12.

NVIDIA Corporation. *NVIDIA DGX B200 Datasheet*. NVIDIA Corporation, 2025a. URL https://resources.nvidia.com/en-us-dgx-systems/dgx-b200-datasheet. Accessed 2026-08-05.

NVIDIA Corporation. *NVIDIA H100 Tensor Core GPU*. NVIDIA Corporation, 2025b. URL https://www.nvidia.com/en-us/data-center/h100/. Accessed 2026-08-05.

NVIDIA Corporation. *NVIDIA HGX Platform*. NVIDIA Corporation, 2025c. URL https://www.nvidia.com/en-us/data-center/hgx/. Accessed 2026-08-05.

NVIDIA Corporation. *CUDA C++ Best Practices Guide*. NVIDIA Corporation, 2026a. URL https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#arithmetic-instructions-throughput-native-arithmetic-instructions. Accessed 2026-07-30.

NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA Corporation, 2026b. URL https://docs.nvidia.com/cuda/cuda-c-programming-guide/. Accessed 2026-08-20.

NVIDIA Corporation. *Parallel Thread Execution ISA, Version 9.4*. NVIDIA Corporation, 2026c. URL https://docs.nvidia.com/cuda/parallel-thread-execution/index.html. Accessed 2026-10-29.

NVIDIA Corporation. *NVIDIA Vera Rubin NVL72*. NVIDIA Corporation, 2026d. URL https://www.nvidia.com/en-us/data-center/vera-rubin-nvl72/. Accessed 2026-08-05.

NVIDIA Corporation. *TensorRT-LLM*. NVIDIA Corporation, 2026e. URL https://github.com/NVIDIA/TensorRT-LLM. Accessed 2026-08-19.

Park, G., Park, B., Kim, M., Lee, S., Kim, J., Kwon, B., Kwon, S. J., Kim, B., Lee, Y., and Lee, D. LUT-GEMM: quantized matrix multiplication based on luts for efficient inference in large-scale generative language models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=gLARhFLE0F.

Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. GPQA: A graduate-level google-proof q&a benchmark. *CoRR*, abs/2311.12022, 2023. doi: 10.48550/ARXIV.2311.12022. URL https://doi.org/10.48550/arXiv.2311.12022.

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/7ede97c3e082c6df10a8d6103a2eebd2-Abstract-Conference.html.

Xia, H., Wu, X., Li, J., Wu, R., Wang, J., Wang, J., Li, C., Singhal, A., Shah, A. D., Ariyak, A., Zhuang, D., Zhou, Z., Athiwaratkun, B., Zheng, Z., and Song, S. L. Kitty: Accurate and efficient 2-bit KV cache quantization with dynamic channel-wise precision boost. *CoRR*, abs/2511.18643, 2025. doi: 10.48550/ARXIV.2511.18643. URL https://doi.org/10.48550/arXiv.2511.18643.

Yao, Z., Aminabadi, R. Y., Zhang, M., Wu, X., Li, C., and He, Y. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28*

- *December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/adf7fa39d65e2983d724ff7da57f00ac-Abstract-Conference.html.

Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. Flashinfer: Efficient and customizable attention engine for LLM inference serving. In Zaharia, M., Joshi, G., and Lin, Y. C. (eds.), *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025*. OpenReview.net/mlsys.org, 2025. URL https://openreview.net/forum?id=RXPofAsL8F.

Zadouri, T., Hoehnerbach, M., Shah, J., Liu, T., Thakkar, V., and Dao, T. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. *CoRR*, abs/2603.05451, 2026. doi: 10.48550/ARXIV.2603.05451. URL https://doi.org/10.48550/arXiv.2603.05451.

Zhang, J., Huang, H., Zhang, P., Wei, J., Zhu, J., and Chen, J. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net, 2025a. URL https://proceedings.mlr.press/v267/zhang25ae.html.

Zhang, J., Wei, J., Wang, H., Zhang, P., Xu, X., Huang, H., Jiang, K., Chen, J., and Zhu, J. Sageattention3: Microscaling FP4 attention for inference and an exploration of 8-bit training. In Belgrave, D., Zhang, C., Montoya, L. N., Lin, H., Pascanu, R., Koniusz, P., Ghassemi, M., Chen, N., Ruíz, I. V. M., and Loaiza-Bonilla, A. (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*, 2025b. URL http://papers.nips.cc/paper_files/paper/2025/hash/4db397e0f760cc573c681e81a01a3dba-Abstract-Conference.html.

Zhang, J., Wei, J., Zhang, P., Zhu, J., and Chen, J. Sageattention: Accurate 8-bit attention for plug-and-play inference acceleration. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025c. URL https://openreview.net/forum?id=OL44KtasKc.

Zhao, Y., Lin, C., Zhu, K., Ye, Z., Chen, L., Zheng, S., Ceze, L., Krishnamurthy, A., Chen, T., and Kasikci, B. Atom: Low-bit quantization for efficient and accurate LLM serving. In Gibbons, P. B., Pekhimenko, G., and Sa, C. D. (eds.), *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*. mlsys.org, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/5edb57c05c81d04beb716ef1d542fe9e-Abstract-Conference.html.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C. W., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

Zhou, Z., Zhuang, D., Li, J., Chen, Z., Song, S. L., Athiwaratkun, B., and Wu, X. OSCAR: offline spectral covariance-aware rotation for 2-bit KV cache quantization. *CoRR*, abs/2605.17757, 2026. doi: 10.48550/ARXIV.2605.17757. URL https://doi.org/10.48550/arXiv.2605.17757.

A KV Packing and Dataflow

Fig. 6 illustrates how FOSCAR packs INT2 K values and constructs Tensor Core operands on Hopper and Blackwell. The same approach extends to INT2 V and INT4 K and V because cache packing arranges quantized values to match each Tensor Core operand layout, and the decode kernel constructs floating-point operands with shifts and masks. Quantizing K or V along tokens or channels changes how scales and offsets are grouped and where affine folding applies them (Sec. 4.1), whereas the integer packing below depends only on the stored bit width and matrix operand layout.

A.1 Hopper: packing for register operands

Hopper packs K and V according to the register fragments that each consumer thread supplies to `wgmma`. For QK, the K operand has token-by-channel coordinates, as shown in Fig. 6a. For PV, the V operand has channel-by-token coordinates, so FOSCAR packs V values according to the transposed V tile. Both layouts let each consumer thread load contiguous packed words from shared memory and construct its register operands without gathering individual values.

Each 32-bit packed word places the two quantized values for a pair of BF16 operands in fields sixteen bits apart. An INT2 word holds sixteen values, forming eight pairs; an INT4 word holds eight values, forming four pairs. A shift aligns both fields with their BF16 mantissas, and a masked merge combines them with a fixed BF16 bit pattern. The stored bit width sets the field spacing and the number of pairs per word. K and V differ only in the logical coordinates of those pairs and share the same pairwise register construction. Affine folding compensates for the resulting operand mapping through Eq. (5).

The Hopper dataflow in Fig. 6b therefore also applies to INT2 V and INT4 K and V. TMA copies the packed tiles from global to shared memory. Consumer threads construct BF16 operands in registers, and all four warps of a consumer warpgroup collectively issue `wgmma`. The QK product consumes K fragments, and the PV product consumes the transposed V fragments.

A.2 Blackwell: packing for shared-memory operands

The Blackwell kernel uses the same packed-word layout for K and V at each stored bit width. Both tensors store channels of a token together, with the values interleaved within each word for direct operand construction. The preparation warpgroup writes the constructed operands to the shared-memory layouts required by QK and PV.

INT2 K and V. Each 32-bit word holds sixteen integer values in eight four-bit groups. Group j places q_j in its low two bits and q_{j+8} in its high two bits, for $j = 0, \ldots, 7$. For a packed word w, the kernel constructs two operand words:

$$u_0 = w \,\&\, \texttt{0x33333333},$$
$$u_1 = (w \gg 2) \,\&\, \texttt{0x33333333}.$$

Each output holds eight E2M1 operands, with the unused bits of each four-bit operand set to zero. The patterns for $q \in \{0, 1, 2, 3\}$ represent $y \in \{0, 0.5, 1, 1.5\}$, giving $y = q/2$. Affine folding compensates with $s' = 2s$ and $z' = z$. The two masks and one shift construct sixteen operands without numeric conversion, accounting for the INT2 cost in Table 1. This is the Blackwell construction shown for K in Fig. 6a; V uses the same bit operations.

INT4 K and V. Each 32-bit word holds eight integer values in four bytes. Byte j places q_j in its low four bits and q_{j+4} in its high four bits, for $j = 0, \ldots, 3$. The corresponding construction is

$$u_0 = w \,\&\, \texttt{0x0F0F0F0F},$$
$$u_1 = (w \gg 4) \,\&\, \texttt{0x0F0F0F0F}.$$

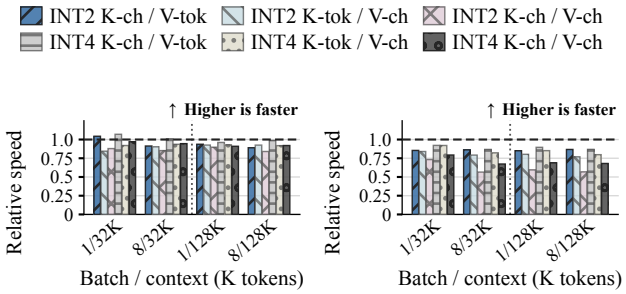
Each output holds four E4M3 operands, with each integer in the low four bits of its byte. For $q \in \{0, \ldots, 15\}$, these patterns represent $y = q \cdot 2^{-9}$. Affine folding compensates with $s' = 512s$ and $z' = z$. The two masks and one shift construct eight operands, accounting for the INT4 cost in Table 1. Marlin W4A8 uses the same INT4-to-E4M3 mapping for weights (Lin, 2025); FOSCAR applies it to unsigned KV values, whose construction needs no sign extraction.

Seven of the sixteen INT4 bit patterns represent subnormal E4M3 values, for which the PTX specification leaves matrix-operand behavior undefined (NVIDIA Corporation, 2026c). We test these subnormal values on B200 across both matrix-operand positions and both shared-memory layouts used by the kernel, and all matrix-instruction outputs match FP64 reference results.

For both stored bit widths, the Blackwell dataflow follows Fig. 6b. TMA copies packed K and V tiles to shared memory, the preparation warpgroup constructs the QK and PV operands in shared memory, and a separate issue thread launches `tcgen05`. The operand format changes from E2M1 for INT2 to E4M3 for INT4, while the division of work across copying, operand preparation, and matrix-product issue remains the same.

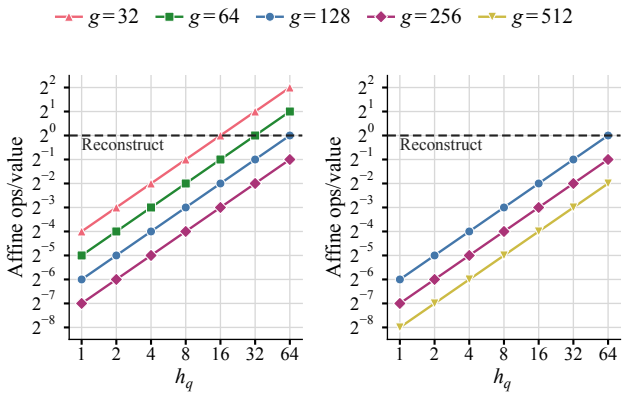
B Quantization-Axis Combinations

The main evaluation uses token-wise K and token-wise V with group size $g = 128$, which applies acc-fold to QK and operand-fold to PV (Sec. 4.1). To evaluate all four



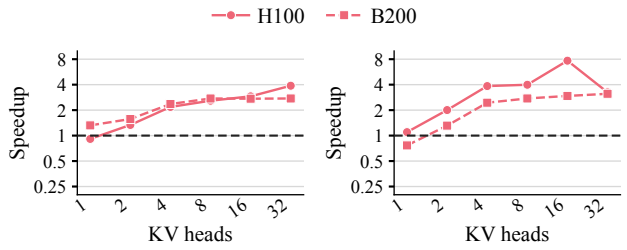
(a) H100. (b) B200.

Figure 13. Decode kernel speed across channel-wise K and V quantization combinations relative to token-wise K and V (1.0, dashed line) at the same stored bit width and shape ($H_Q = 32$, $H_{KV} = 8$, $D = 128$, group size $g = 128$).



(a) Token-wise quantization. (b) Channel-wise quantization.

Figure 14. Scalar scale and offset operations per stored K or V value for affine folding versus element-wise $sq + z$ reconstruction (dashed).



(a) Batch 1. (b) Batch 8.

Figure 15. FOSCAR INT2 speedup over FlashInfer BF16 across H_{KV} at 128K context ($H_Q = 32$, $D = 128$).

combinations of token-wise and channel-wise K and V, we benchmark FOSCAR INT2 and INT4 on H100 and B200 across $B \in \{1, 8\}$ and $KV \in \{32\text{K}, 128\text{K}\}$ at $H_Q = 32$, $H_{KV} = 8$, $D = 128$, and $g = 128$, following the timing methodology of Sec. 6 and verifying every output against an FP32 reference.

Fig. 13 compares the decode speed of the three channel-wise K and V combinations against token-wise K and V (1.00) at the same stored bit width and shape. On H100 (Fig. 13a), the channel-wise combinations run at 0.84–1.04× the speed of token-wise K and V at INT2 and 0.91–1.07× at INT4, corresponding to 2.08–4.00× (INT2) and 1.75–3.35× (INT4) speedups over FlashInfer BF16 across all four axis combinations. On B200 (Fig. 13b), the single-axis channel-wise combinations (*K-ch / V-tok* and *K-tok / V-ch*) run at 0.77–0.87× the speed of token-wise K and V at INT2 and 0.80–0.92× at INT4, while channel-wise K and V (*K-ch / V-ch*) runs at 0.57–0.73× at INT2 and 0.67–0.79× at INT4, corresponding to 1.50–2.74× (INT2) and 1.50–2.36× (INT4) speedups over FlashInfer BF16 across all four combinations. Channel-wise K and channel-wise V read more scales and offsets and perform per-group work along the token dimension: operand-fold on channel-wise K scales Q for each token group, while acc-fold on channel-wise V rescales group-local PV partial results across token groups. The relative gap is larger on B200 than on H100 for two architectural reasons: first, the Blackwell kernel tiles the sequence at 256 tokens rather than 64, so $g = 128$ splits each sequence tile into two sub-tile groups and runs both the Q'-plane builder and the per-group PV accumulator drain inside the correction warpgroup; second, Blackwell's `tcgen05` QK product consumes an FP8 E4M3 query operand rather than Hopper's BF16 operand, so each channel-wise K group must compute a per-row maximum and power-of-two scale to requantize Q' to E4M3. With KIVI-style channel-wise K and token-wise V (Liu et al., 2024; Hooper et al., 2024), FOSCAR runs at 0.89–1.04× of token-wise speed on H100 (2.42–3.56× over FlashInfer BF16) and 0.85–0.87× on B200 (2.11–2.38× over FlashInfer BF16) at INT2, and 0.96–1.07× on H100 (2.03–3.31× over FlashInfer BF16) and 0.87–0.92× on B200 (1.94–2.05× over FlashInfer BF16) at INT4.

C Head-Ratio and Group-Size Trade-off

When $h_q = H_Q/H_{KV}$ query heads share one KV head and each quantization group spans g values, affine folding performs one scale multiplication and one offset multiply-add per group for each query head, or $2h_q/g$ scalar affine operations per stored value. Reconstructing each cached value once with $sq + z$ and reusing it across the h_q query heads costs one multiply-add per stored value. Fig. 14 compares the scalar affine operations per value as h_q and g vary. At $h_q = 4$ and $g = 128$, affine folding uses $\frac{2\times4}{128}$ operations per value, a 16× reduction. Break-even occurs at $g = 2h_q$; because standard grouped-query attention models use group sizes $g \geq 32 > 2h_q$, affine folding requires up to 64× fewer scalar affine operations than per-element reconstruction.

Fig. 15 measures this trade-off empirically by varying H_{KV} from 1 to 32 (h_q from 32 down to 1) with $H_Q = 32$, $D = 128$, and $g = 128$ at 128K context for batches 1 and 8. At $H_{KV} = 1$ ($h_q = 32$), the smaller KV cache and higher $2h_q/g$ ratio limit the gain over FlashInfer BF16 ($0.91\times$ and $1.10\times$ on H100; $1.32\times$ and $0.77\times$ on B200). As H_{KV} grows, KV traffic increases while h_q decreases, raising FOSCAR INT2's speedup over FlashInfer BF16 to $3.89\times$ at batch 1 and $7.68\times$ at batch 8 on H100, and to $2.75\times$ and $3.12\times$ on B200 (Figs. 15a and 15b).